

Gerrit Merge Conflict Resolver

Maintainer's Manual

CS 5150 — Group 17 — Spring 2026

1. System Overview

The Gerrit Merge Conflict Resolver is a frontend-only extension to Gerrit Code Review that enables developers to resolve merge conflicts directly in the browser. It introduces a three-panel merge editor (`gr-merge-editor`) that is activated when a Gerrit change contains git conflict markers. The implementation deliberately reuses Gerrit's existing inline-edit infrastructure for saving resolutions as new patch sets, adding no new backend endpoints or services.

The feature lives entirely within the `polygerrit-ui` layer (Gerrit's Polymer-based frontend) and is activated via existing Gerrit API metadata fields (`contains_git_conflicts`, `RevisionInfo.conflicts`).

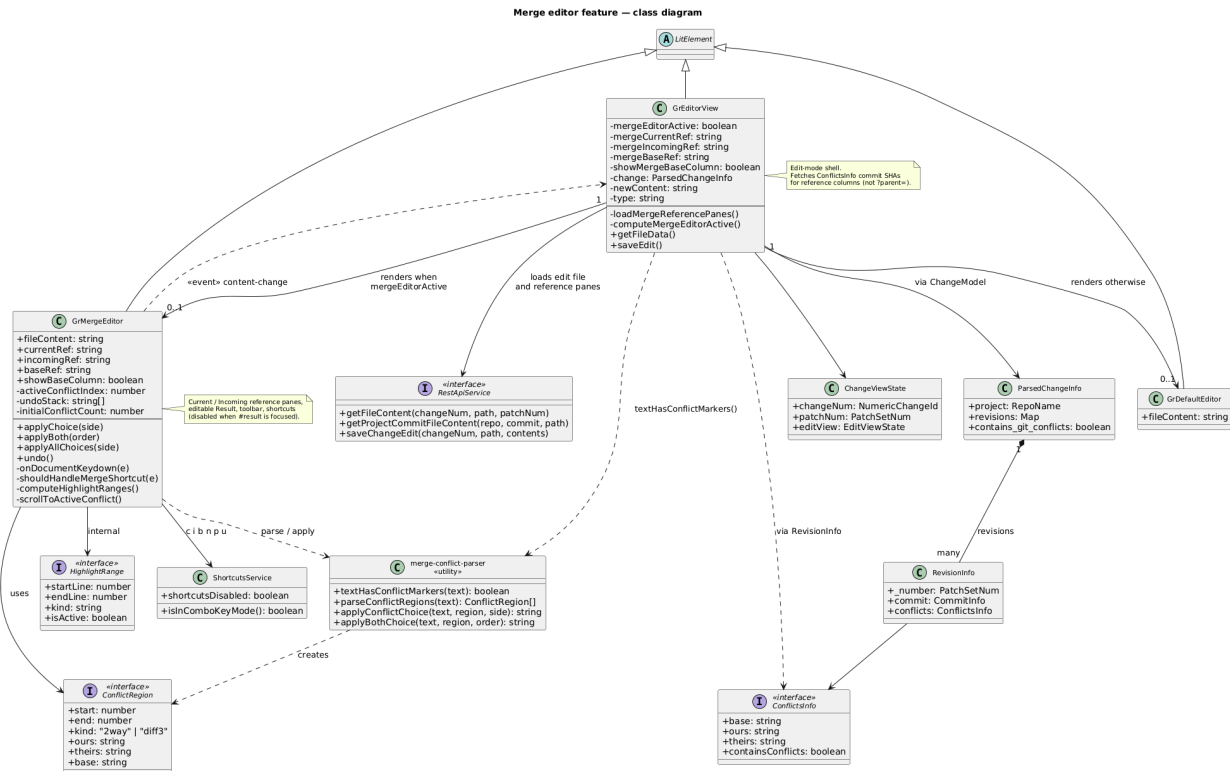
2. Repository and Branch Structure

- Repository: <https://github.com/autoelk/gerrit-merge-conflicts>
- Implementation branch: `three-panel-merge` (12 commits ahead of master)
- All delivered code is merged to `three-panel-merge`. Do not reference the master branch for implementation code.

Key paths within the repository:

Path	Contents
<code>polygerrit-ui/app/elements/</code>	Root of all Polymer web component source files.
<code>polygerrit-ui/app/elements/edit/gr-merge-editor/</code>	Primary merge editor component (<code>gr-merge-editor</code>).
<code>polygerrit-ui/app/elements/edit/gr-editor-view/</code>	Activation and data-loading logic for the editor view.
<code>polygerrit-ui/app/utils/merge-conflict-parser/</code>	Conflict marker parser (<code>merge-conflict-parser</code>).
<code>polygerrit-ui/app/elements/edit/gr-merge-editor/*_test.ts</code>	Unit and integration tests for the merge editor.

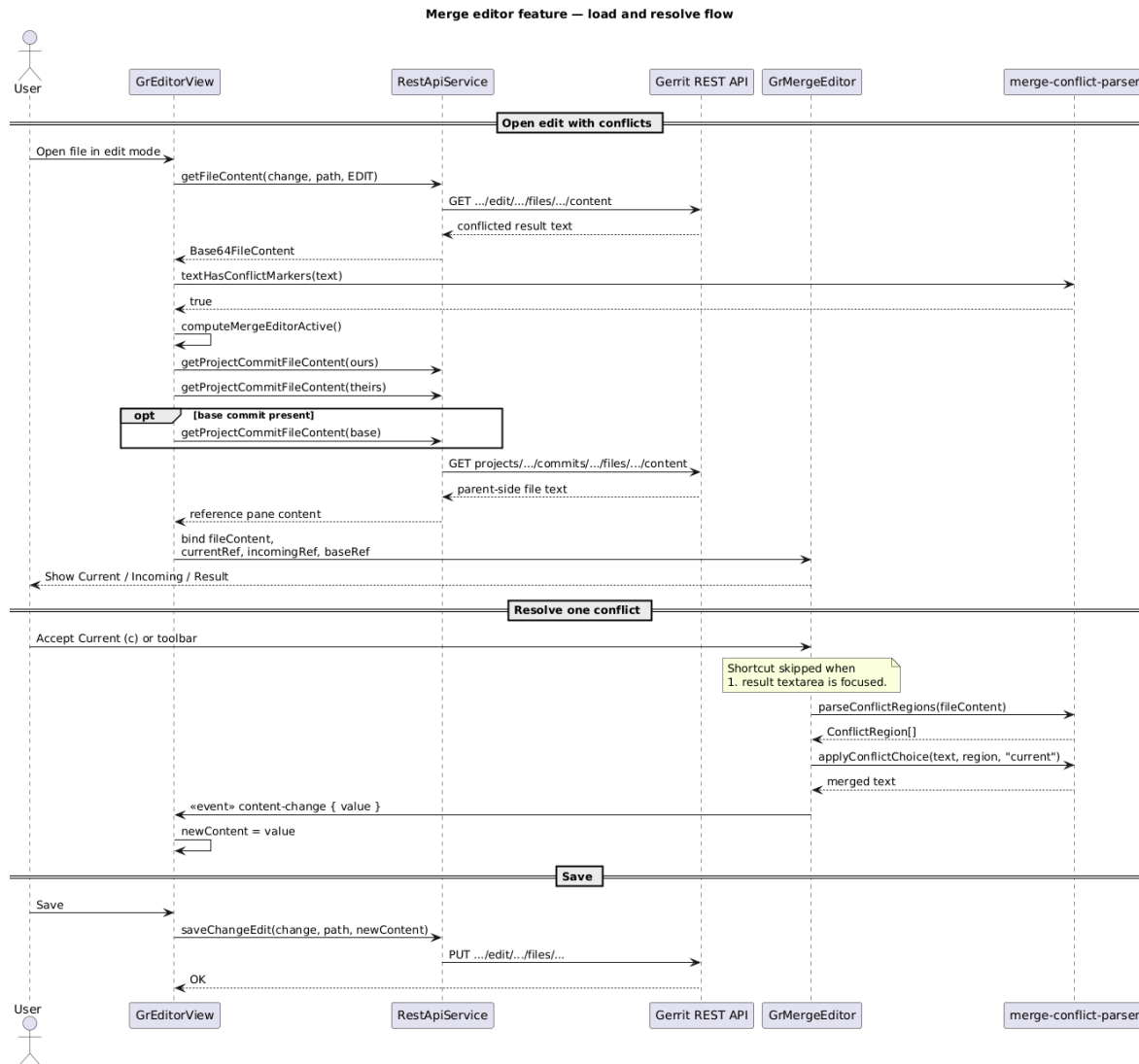
3. Architecture



3.1 Component Overview

The implementation follows Gerrit's standard Polymer web component architecture. All components are written in TypeScript and use LitElement/Polymer patterns consistent with the rest of polygerrit-ui.

Component / Module	Responsibility
gr-merge-editor	The main three-panel UI component. Renders Current, Incoming, and Result panes; handles resolution actions (accept current/incoming/both/all), undo stack, navigation, progress tracking, word wrap, and dark mode.
gr-editor-view	Activates the merge editor when a conflicted change is detected. Loads file content from Gerrit API and passes conflict data to gr-merge-editor.
merge-conflict-parser	Parses raw file content containing git conflict markers (both two-way <<<<<<< / ===== / >>>>>>> and diff3 formats) into structured conflict objects consumed by gr-merge-editor.
Gerrit Inline Edit Flow	Not a new component — the existing Gerrit save/publish pipeline reused to commit resolved content as a new patch set. No new backend code was added.



3.2 Data Flow

The high-level data flow for a conflict resolution session is:

1. gr-editor-view detects `contains_git_conflicts = true` on the current revision via the Gerrit REST API.
2. gr-editor-view fetches the raw file content (including conflict markers) for the conflicted file and the reference commit IDs.
3. merge-conflict-parser parses the raw content, returning an array of conflict objects, each containing current lines, incoming lines, and surrounding context.
4. gr-merge-editor renders the three panels from the parsed conflict data. The Result pane is pre-populated with the raw file content (including markers) as the starting point.
5. As the user accepts resolutions, gr-merge-editor updates the Result pane content and the internal conflict state, removing markers and updating the progress counter.
6. On save, the Result pane content is passed back to gr-editor-view, which submits it via Gerrit's inline-edit API, creating a new patch set.

3.3 Key API Fields

- `contains_git_conflicts` (boolean on `RevisionInfo`): Set to true by Gerrit when a revision contains git conflict markers. Used by `gr-editor-view` to decide whether to activate the merge editor.
- `RevisionInfo.conflicts`: Metadata about conflicted files in the revision. Used to identify which files need resolution.
- Inline Edit API (`/a/changes/{id}/edit`): Standard Gerrit endpoint used (unchanged) to save the resolved file content as a new edit, which is then published as a patch set.

4. Requirements Summary

4.1 Functional Requirements (Delivered)

- FR-1: The system shall display a three-panel merge editor (Current, Incoming, Result) for any Gerrit change containing git conflict markers.
- FR-2: The system shall highlight conflicting regions with distinct colors per branch.
- FR-3: The system shall support resolution actions: Accept Current, Accept Incoming, Accept Both, Accept All.
- FR-4: The system shall provide an undo action that reverses resolution decisions one step at a time.
- FR-5: The system shall provide navigation between conflicts within a file (next/previous).
- FR-6: The system shall display a progress indicator showing resolved vs. total conflicts.
- FR-7: The system shall allow direct editing of the Result pane.
- FR-8: The system shall save the resolved content as a new Gerrit patch set via the existing inline-edit flow.
- FR-9: The system shall support dark mode and word-wrap toggles.
- FR-10: The system shall provide keyboard shortcuts for all resolution actions.
- FR-11: The system shall support both two-way and diff3 conflict marker formats.

4.2 Functional Requirements (Not Delivered / Deferred)

- FR-D1: Multi-file conflict navigation (full file system view). Not started.
- FR-D2: Complete base-column (common ancestor) wiring. Partially implemented.
- FR-D3: CodeMirror integration for the Result pane. Not started.
- FR-D4: Re-parse-after-mutation (re-parsing conflict state after direct Result pane edits). Not started.

4.3 Non-Functional Requirements

- NFR-1: The implementation shall not introduce new backend services or REST endpoints.
- NFR-2: The UI shall follow Gerrit's existing design language and style conventions.
- NFR-3: All new components shall be covered by unit tests (target: 95% UI coverage).
- NFR-4: All new functions and test cases shall be documented inline.

5. Development Environment Setup

5.1 Prerequisites

- Node.js v18 or later
- Java 17 (required by Bazel and Gerrit's build system)
- Bazel 6.x (or use the Bazelisk wrapper included in the repository)
- A Unix-like environment (Linux or macOS). Windows with WSL2 is supported.

5.2 Cloning and Branching

```
git clone https://github.com/autoelk/gerrit-merge-conflicts.git
cd gerrit-merge-conflicts
git checkout three-panel-merge
```

5.3 Installing Frontend Dependencies

```
cd polygerrit-ui
npm install
```

5.4 Running the Frontend Dev Server

Gerrit's frontend can be developed against a live Gerrit backend or using a mock server. To run the polygerrit-ui dev server:

```
cd polygerrit-ui
npm run start
```

This starts a local dev server at `http://localhost:8080`. You will need a running Gerrit backend (see Section 5.5) for the merge editor to function end-to-end.

5.5 Running a Local Gerrit Instance (for E2E Testing)

End-to-end testing requires a live Gerrit instance with a real conflicted change. The recommended approach:

7. Build Gerrit from source using Bazel: `bazel build release`
8. Run the resulting WAR: `java -jar bazel-bin/release.war init --batch --dev -d /tmp/gerrit_test`
9. Start Gerrit: `/tmp/gerrit_test/bin/gerrit.sh start`
10. Create a test repository, make two diverging commits on separate branches, and rebase one onto the other to produce a real merge conflict.
11. Push the conflicted change to your local Gerrit instance and verify that `contains__git__conflicts` is set on the revision.

Note: *Setting up a Gerrit instance from scratch is time-consuming. The team recommends maintaining a persistent test instance with pre-staged conflicted changes for repeated E2E testing.*

6. Test Facilities

6.1 Unit Tests

Unit tests are written in TypeScript using Gerrit's standard test harness (based on Mocha/Chai with Web Test Runner). Test files are co-located with their source files and named `*_test.ts`.

To run all unit tests:

```
cd polygerrit-ui
npm test
```

To run tests for a specific component:

```
npm test -- --grep gr-merge-editor
```

Current coverage: 95% of UI component code. The test suite covers:

- merge-conflict-parser: parsing of two-way and diff3 markers, edge cases (empty blocks, adjacent conflicts, overlapping edits).
- gr-merge-editor: resolution actions (accept current/incoming/both/all), undo stack correctness, progress counter updates, fallback behavior when reference pane fails to load.
- Navigation: next/previous conflict, keyboard shortcut handlers.

6.2 End-to-End Tests

E2E tests require a running Gerrit instance with a staged conflicted change (see Section 5.5). E2E tests use Gerrit's existing Selenium/WebDriver test infrastructure.

To run E2E tests:

```
bazel test //polygerrit-ui/app/elements/edit/gr-merge-editor:e2e_test
```

Note: *E2E tests are the primary gap identified in client feedback. Priority for future contributors is to expand E2E coverage to include: the full accept → save → new patch set workflow; edge cases with overlapping and adjacent conflicts; large files; and files with only whitespace-level conflicts.*

6.3 Preventing Regressions

Before merging any change to three-panel-merge:

12. Run the full unit test suite (npm test) and confirm all tests pass.
13. Manually verify the merge editor loads and resolves a conflict on a local Gerrit instance.
14. Run E2E tests if the change touches gr-editor-view, the save/publish path, or merge-conflict-parser.
15. Close the corresponding GitHub issue for the change.

7. Developer Workflow and Style Guide

7.1 Branching

- All new work branches off three-panel-merge.
- Feature branches should be named feature/<short-description> (e.g., feature/codemirror-result-pane).
- Bug fix branches: fix/<short-description>.

7.2 Commit Guidelines

- One logical change per commit.
- Commit messages: imperative mood, present tense (e.g., “Add undo stack to gr-merge-editor”).
- Reference GitHub issues in commit messages where applicable (e.g., “Fixes #12”).

7.3 Code Style

- TypeScript with strict mode enabled. No any types without a justifying comment.

- Follow Gerrit's existing Polymer/LitElement patterns. Consult existing components (e.g., gr-diff, gr-editor-view) as style references.
- All new public methods and non-trivial private methods must have JSDoc comments.
- All new test cases must include a comment describing what is being tested and why.

7.4 Pull Requests

- All changes require at least one team member review before merging.
- PR description must reference the associated GitHub issue.
- Tests must pass before merge. Do not merge with failing tests.

8. Unfinished Work and Follow-Up Items

The following items are the highest-priority tasks for future contributors, ordered by priority:

P r i .	Item	Location	Remaining Work
1	E2E and integration test coverage	polygerrit-ui/app/elements/edit/g r-merge-editor/	Write automated E2E tests covering the full accept → save → publish workflow. Cover edge cases: overlapping edits, adjacent conflicts, large files, whitespace-only conflicts.
2	Base-column (common ancestor) wiring	gr-merge-editor, gr-editor-view	Connect base commit ID to reference-pane data loader. Add visible error/warning if load fails. Validate against real three-way conflicts.
3	Active-conflict highlighting polish	gr-merge-editor, merge-conflict-parser	Implement scroll-to-active-conflict on navigation. Add stronger visual emphasis (border or background) to the currently focused conflict block.
4	Re-parse-after-mutation	merge-conflict-parser, gr-merge-editor	Trigger conflict re-parse when the user edits the Result pane directly, to keep progress state consistent.
5	CodeMirror for Result pane	gr-merge-editor	Evaluate and integrate CodeMirror to provide syntax highlighting and a richer editing experience in the Result pane.

9. Deployment

This feature is implemented as part of polygerrit-ui and ships with the Gerrit frontend build. There is no separate deployment step for the merge editor itself.

9.1 Building for Deployment

```
bazel build release
```

The resulting release.war includes the compiled polygerrit-ui with the merge editor. Deploy to a Gerrit instance by replacing the WAR and restarting.

9.2 Feature Activation

The merge editor activates automatically when Gerrit sets `contains_git_conflicts = true` on a revision. No feature flags or configuration changes are required. The feature is available to all users who have edit access to a conflicted change.

9.3 Upstreaming to Gerrit

This implementation is intended to eventually be submitted upstream to the main Gerrit project (googlesource.com/gerrit). Before submitting upstream:

- E2E test coverage must be substantially improved (see Section 8, priority 1).
- Base-column wiring should be completed or explicitly removed.
- The change must pass Gerrit's upstream CI (based on Bazel + Java tests + polygerrit-ui tests).
- Code must be submitted via Gerrit's own review process (i.e., pushed to the upstream Gerrit instance for review by Gerrit maintainers).